



:RadioStationsList ארכיטקטורה של נגן אודיו גלובלי

מקרה בוחן הנדסי וניתוח טכני

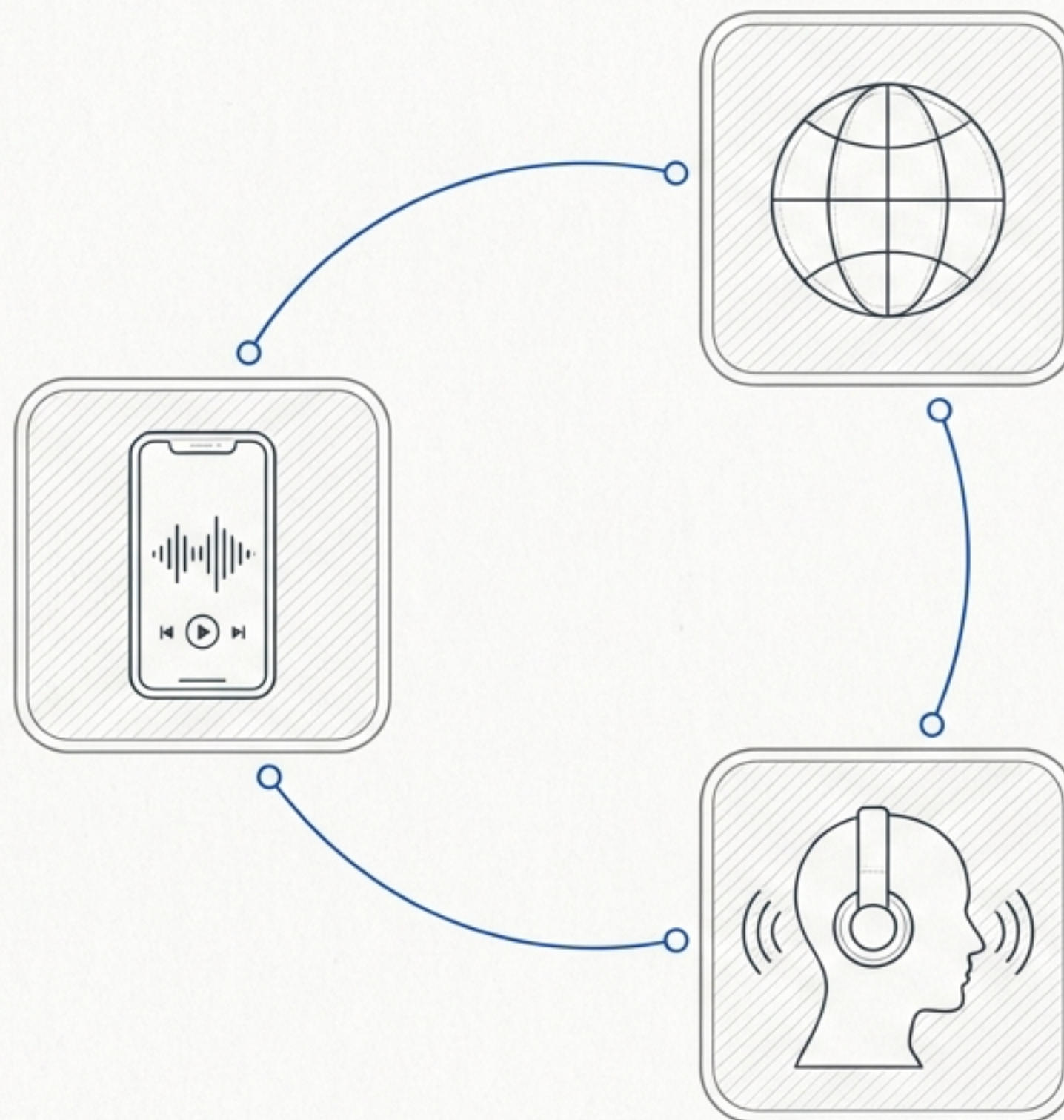
```
/* Author: זאב פריימן (Zeev Fraiman)
/* Date: מאי 2026
/* Tech_Stack: Java, Android Studio
/* Target_API: minSdk 28 / targetSdk 35
```


הבעיה

צורך בגישה ישירה ומהירה
לתחנות רדיו מקוונות ברחבי
העולם (דרך API), ללא מערכות
כבדות ומסורבלות.

קהל היעד

חובבי מוזיקה לפי ז'אנר, צרכני
חדשות עולמיות ומטיילים המחפשים
חיבור תוכן גלובלי.



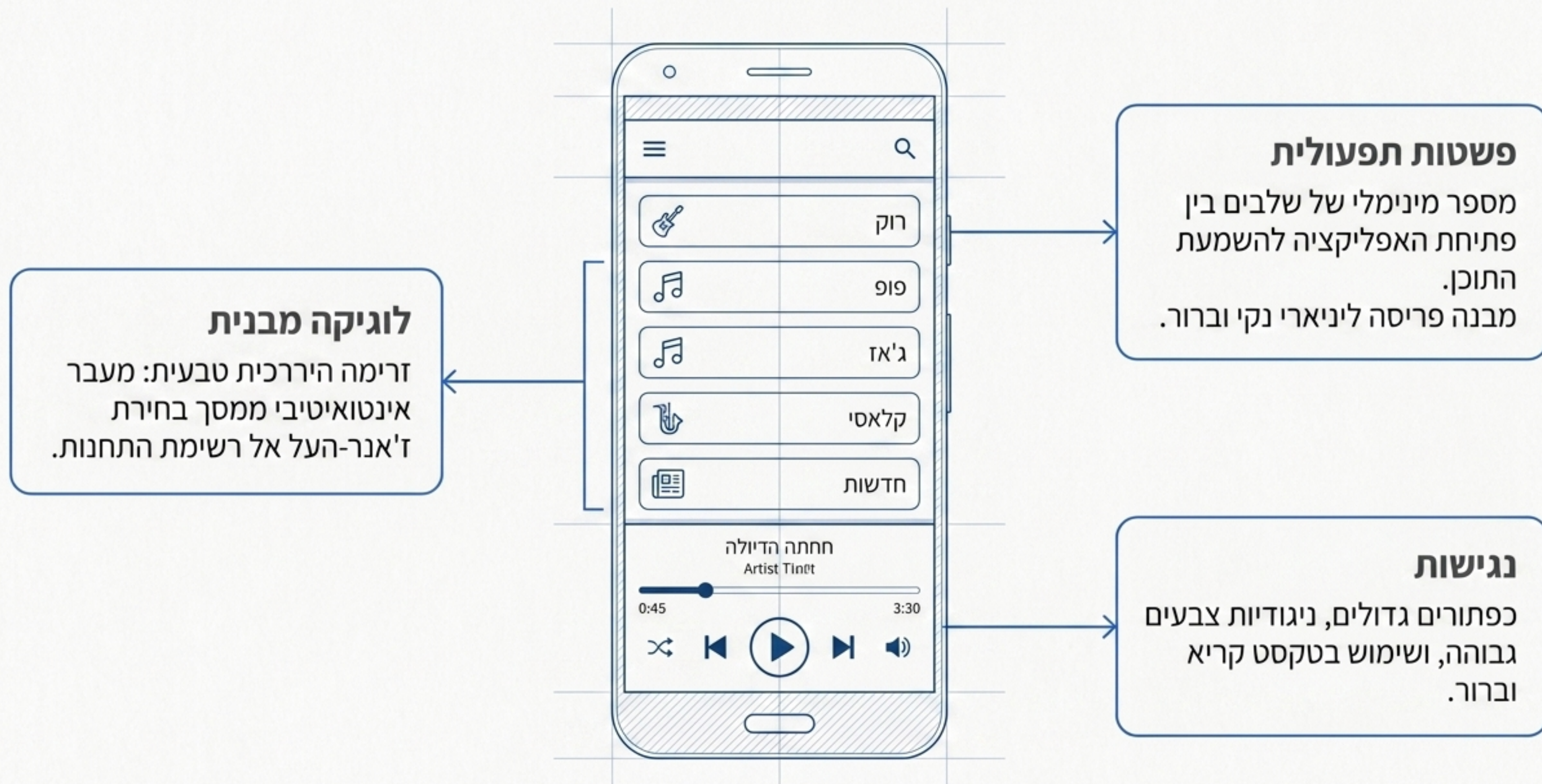
הפתרון

נגן מדיה נייד, קל משקל, התומך
בהשמעה רציפה ברקע תוך
צריכת משאבים מינימלית.

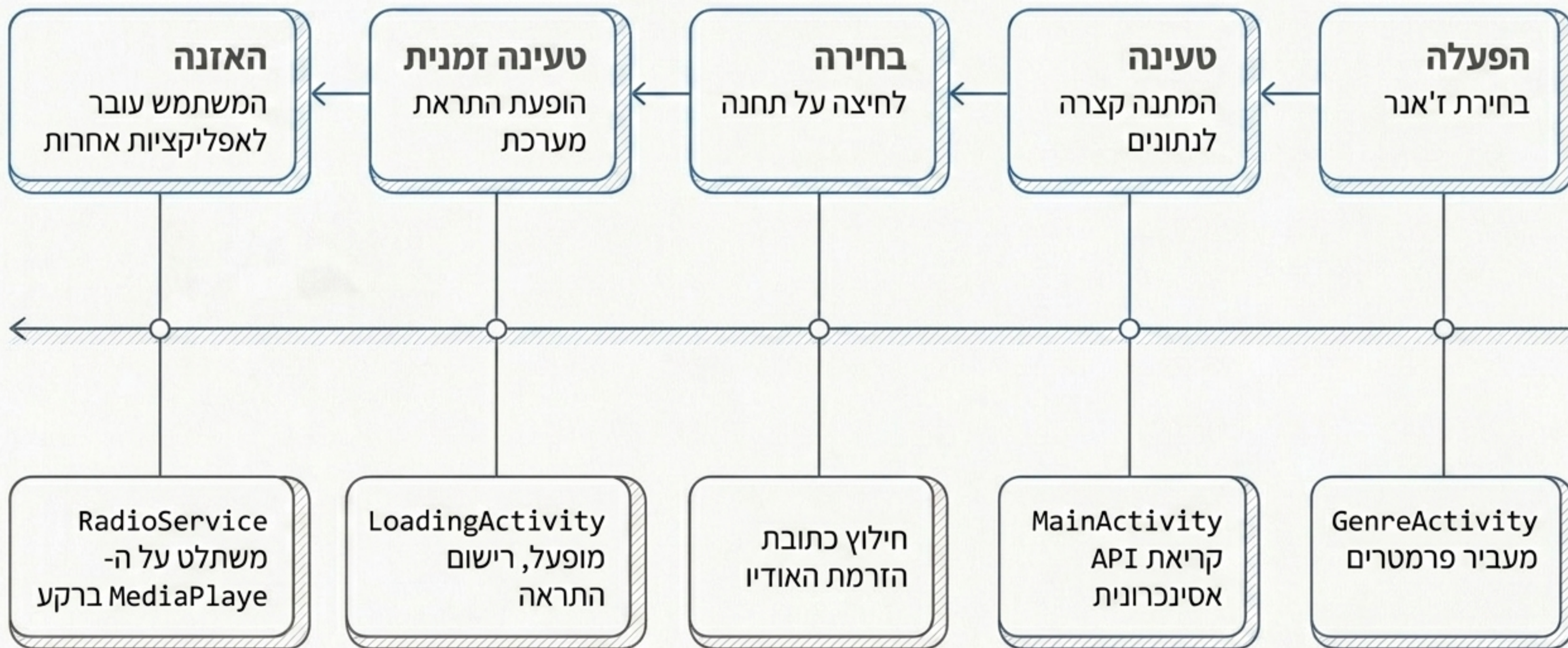
מפרט דרישות המערכת

דרישות לא פונקציונליות	דרישות פונקציונליות
ביצועים: טעינה אסינכרונית מהירה ללא חסימת משתמש	משיכת רשימות תחנות מובילות דרך REST API
חוויית משתמש (UX): ממשק אינטואיטיבי הדורש מינימום קליקים	סינון דינמי לפי קטגוריות וז'אנרים
אמינות: התאוששות שקופה משגיאות רשת וטיפול באגירת נתונים	השמעת אודיו ישירות מתוך סביבת האפליקציה
	ניהול השמעה ברקע באמצעות Service
	הצגת התראות מערכת לניהול סטטוס ההשמעה

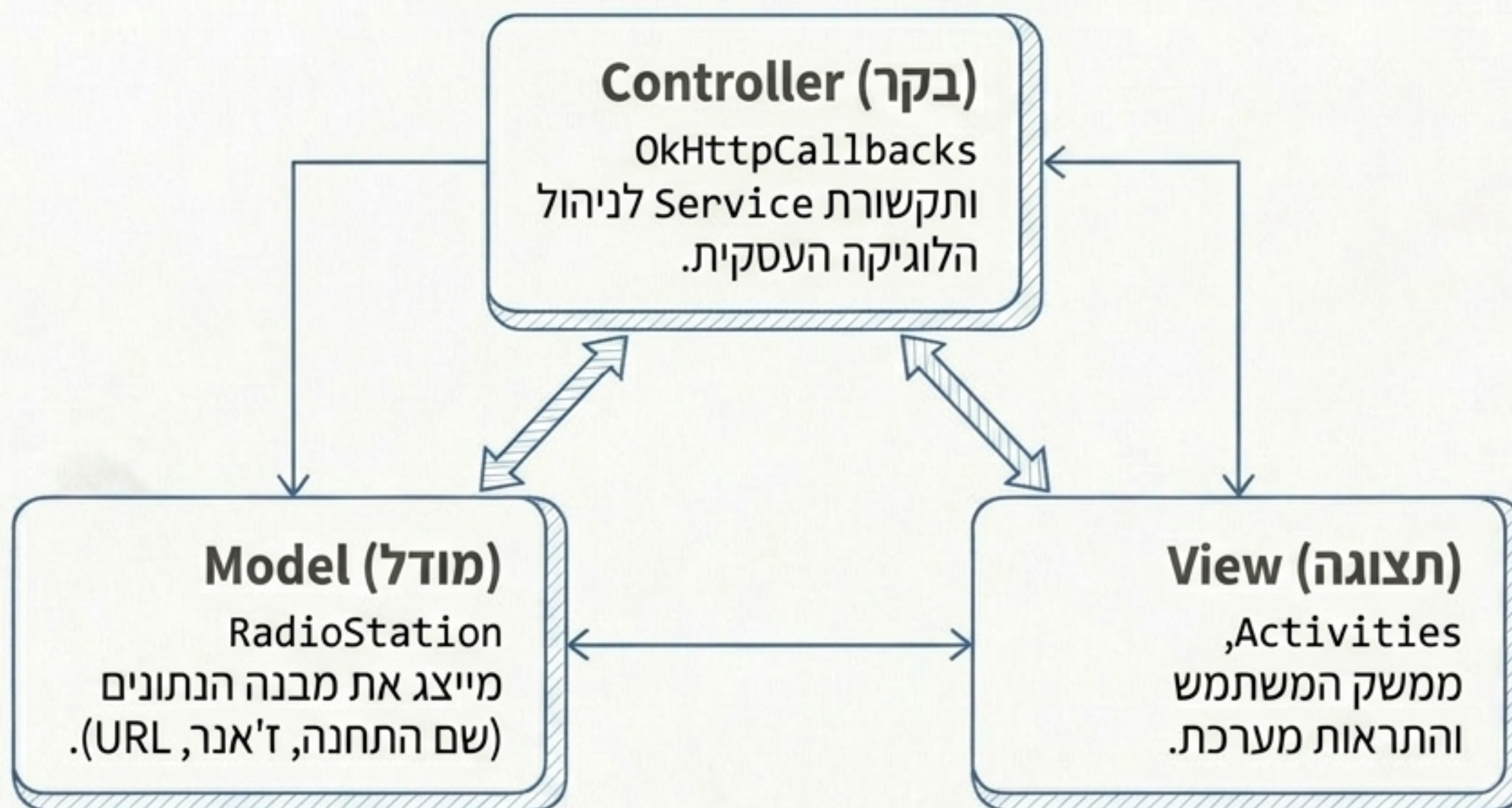
פילוסופיית ה-UX: מינימליזם ונגישות



מסע המשתמש וזרימת הנתונים



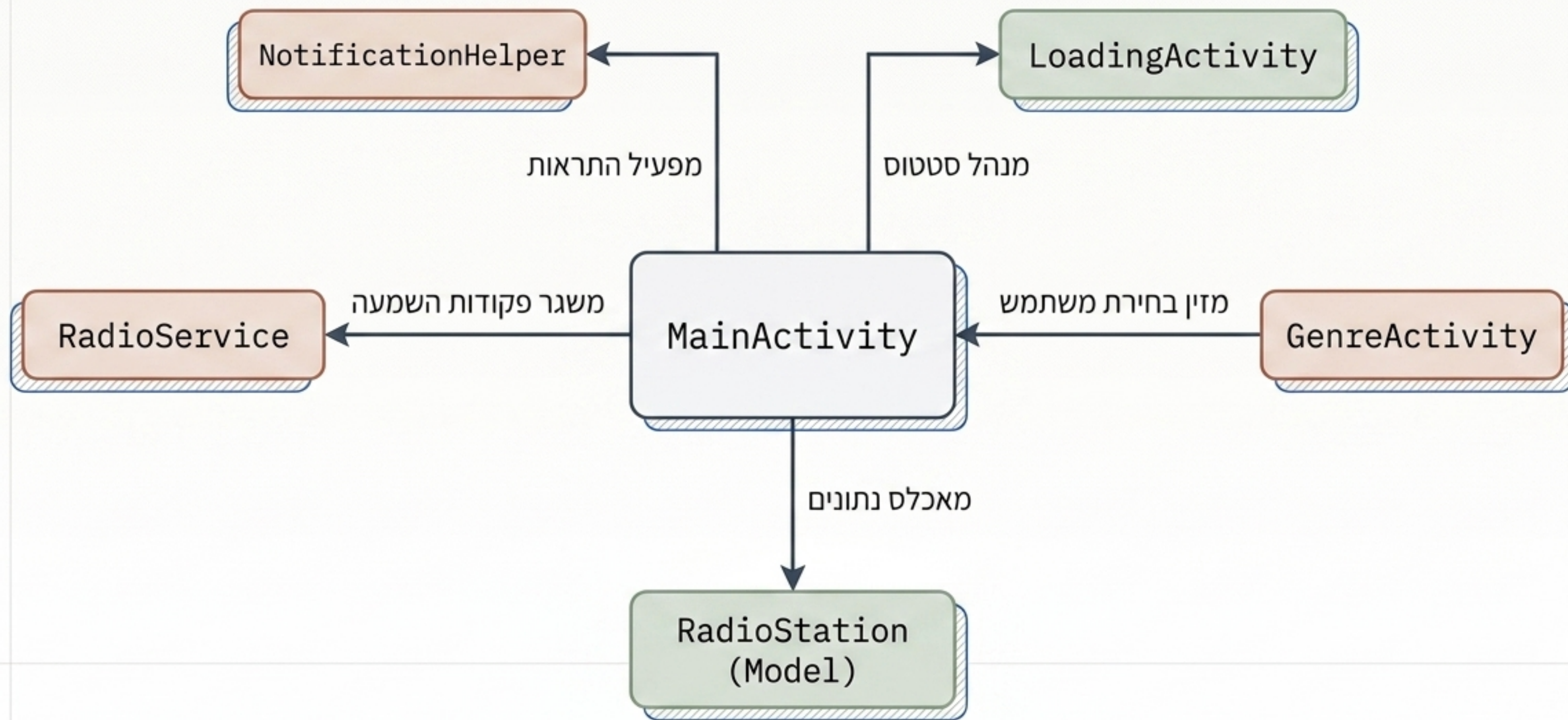
הארכיטקטורה הנבחרת: MVC מותאם אנדרואיד



למה MVC?

פשוטה וישירה עבור אפליקציות Utility. מבטיחה תקורה נמוכה, תגובתיות מהירה וחלוקת תפקידים ברורה.

מפת רכיבי המערכת (UML Topology)



הלב הפועם: חלוקת תפקידים בין תצוגה לרקע

RadioService (מנוע הרקע)

תפקיד: רכיב ביצוע מנותק-תצוגה (Headless)

אחריות: שימור השמעת אודיו רציפה גם כשהאפליקציה נסגרת.

מתודות:
onStartCommand(), onDestroy()

MainActivity (בקר הממשק)

תפקיד: המרכז הפונקציונלי החזותי

אחריות: הצגת רשימות, ייזום בקשות API, שליטה בהשמעה מקומית.

מתודות: onCreate(),
fetchRadioStations(),
playRadio()

ניהול תהליכונים ומניעת ANR

Main UI Thread (תהליכון ראשי)

נשאר פנוי. אחראי אך ורק על אנימציות, לחיצות ועדכוני מסך.



`runOnUiThread()`

`runOnUiThread()`

הזרקת הנתונים
המעובדים חזרה
לממשק בצורה בטוחה.

Background Threads (תהליכונים רקע)

OkHttp Async Callbacks
מנהל קריאות רשת.

Timer Thread
מנהל השהיות.



הגנה על משאבים

מניעת דליפות זיכרון: אובייקטי MediaPlayer וה-Service משוחררים במפורש ב-`onDestroy()`.

צינור הנתונים: מרשת לזיכרון



החלטה הנדסית: מדוע זיכרון פנימי ולא מסד נתונים?

התוכן דינמי. אחסון ב-RAM מספק מהירות גישה מקסימלית וחוסך כתיבות מיותרות לדיסק (I/O) ללא צורך באחסון קבוע.

עמידות המערכת: מנגנוני אל-כשל (Failsafes)

תגובת המערכת וה-UX	תקלת מערכת
בלימת קריסה אסינכרונית, הצגת Toast למשתמש ואיפוס מצב הטעינה. ✓	פסק זמן מול השרת (Network Timeout) ⚠
זיהוי מוקדם במודל, מניעת קריסת MediaPlayer והצגת שגיאה. ✓	כתובת הזרמה (URL) פגומה מה-API ⚠
בדיקת Null מקיפה בכניסה ל-Service ועצירה בטוחה לפני הקצאת משאבים. ✓	הפעלת שירות רקע עם Intent חסר ⚠

מעטפת אבטחה ובדיקות תוכנה

אסטרטגיית בדיקות (Testing)



בדיקות יחידה (Unit Tests): אימות
לוגיקה ומבני נתונים
(ExampleUnitTest)

בדיקות ממשק (UI Tests): אימות
ההקשר וניתוב המחלקות
(ExampleInstrumentedTest)

אבטחה והרשאות (רמה בסיסית)



שימוש בהרשאות אנדרואיד מינימליות:

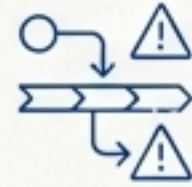
INTERNET: תקשורת מול ה-API והזרמה.

POST_NOTIFICATIONS: עדכון מצב שירות
הרקע.

האפליקציה אינה אוספת נתוני משתמש רגישים.

רטרוספקטיבה: הישגים הנדסיים מול אתגרים

האתגר המורכב ביותר



ניהול מצבי מחזור החיים (Lifecycle).

שמירה על סנכרון MediaPlayer בין מחזור החיים הקצר של ה-Activity לארוך של ה-Service, תוך הימנעות הימנעות מדליפות משאבים.

ההישג המרכזי



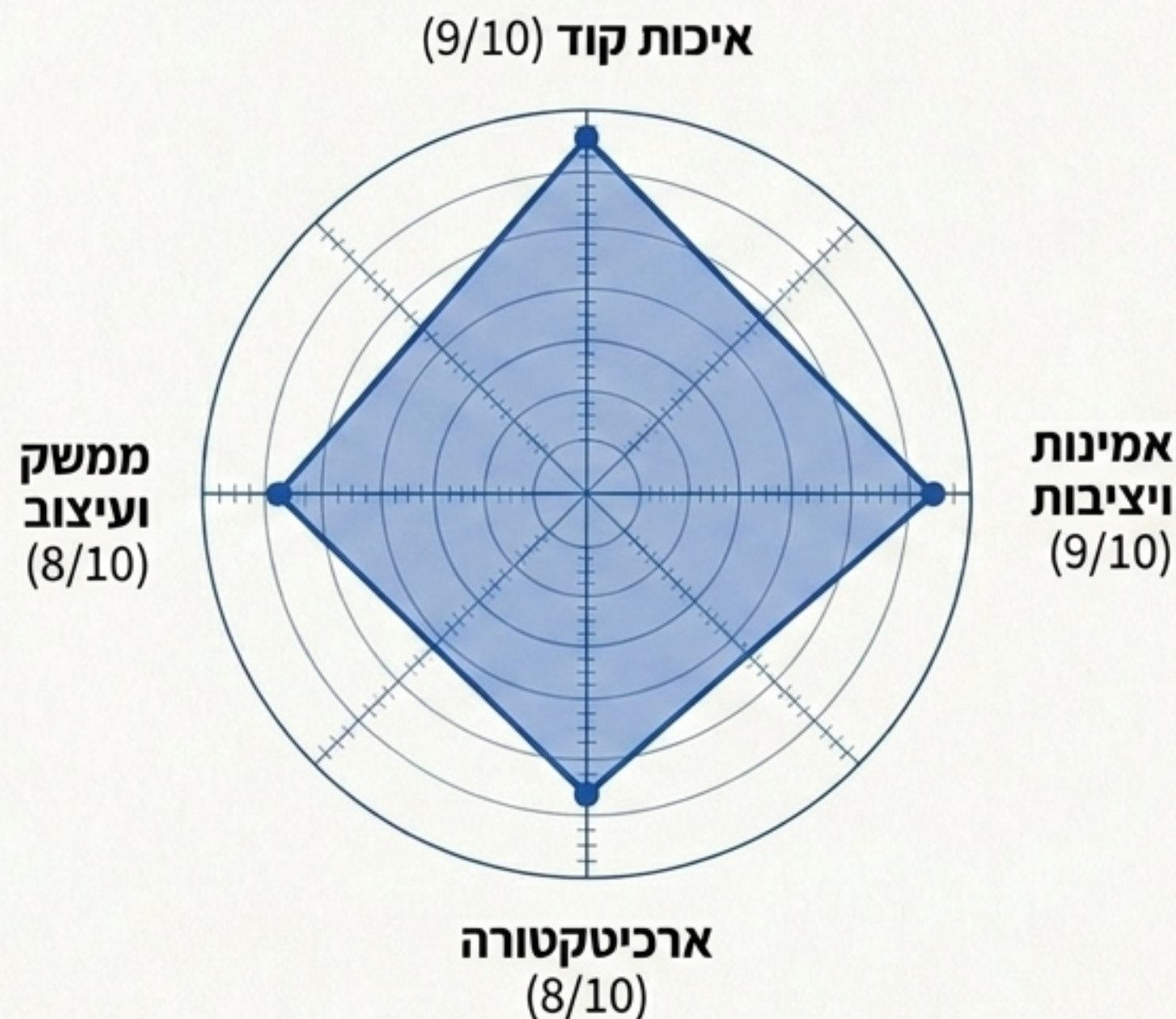
שילוב סינרגי בין שירות רקע להתראות.

יצירת חווית אודיו רציפה באמצעות Service שקוף למשתמש, מלווה בהתראות בעדיפות גבוהה לשליטה מהירה.

מדד איכות: פרופיל מערכת פרופיל מערכת

ציון משוקלל כללי: 10 / 8.5

תובנה: המערכת מפגינה חוזקה
יוצאת דופן בשכבות התשתית,
ניהול השגיאות ואיכות הקוד.
הציון משקף בסיס יציב להרחבות
עתידיות בביטחון מלא.



מפת דרכים: לקראת גרסה 2.0

